

8086 Data Transfer Instructions

1. Quick Overview

Concept

Data transfer instructions move data between registers and memory, or help work with **addresses and pointers**. They are fundamental for loading operands before doing arithmetic/logic.

Exam Tip

Remember: Many 8086 instructions do **not allow memory-to-memory** operations. Usually, at least one operand must be a **register**.

2. MOV (Move)

Concept

MOV copies source data into the destination. Think of it as: *destination gets the same value as source*.

Rules / Notes

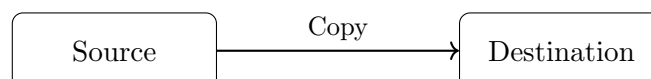
Syntax

```
MOV destination, source
```

Rules

- Source: register / memory / immediate (constant)
- Destination: register or memory
- **Memory-to-memory MOV is not allowed**
- **FLAGS are not affected**

Data Flow (Visual)



Example

```
MOV CX, AX
```

Explanation: Register-to-register move. If $AX = 1234H$, then $CX = 1234H$.

Addressing-Based Examples (Very Important)

Example

```
; Immediate addressing (constant -> register)
MOV AX, 0A25H

; Register addressing
MOV DX, CX

; Direct memory addressing (offset is given)
MOV AX, [2000H]      ; AX <- word at DS:2000H

; Register indirect addressing (offset in register)
MOV BL, [SI]         ; BL <- byte at DS:SI

; Based addressing (base + displacement)
MOV AX, [BX + 04H]   ; EA = BX + 04H

; Indexed addressing (index + displacement)
MOV DL, [DI + 10H]   ; EA = DI + 10H
```

Explanation (Short & Exam-friendly):

- **Immediate:** loads constant directly into register (no memory access).
- **Direct memory:** offset is written in instruction. Physical address:

$$PA = DS \times 10H + \text{offset}$$

- **Indirect/Based/Indexed:** CPU computes **Effective Address (EA)** first, then reads memory using DS by default.

3. XCHG (Exchange)

Concept

XCHG swaps two operands. No programmer-visible temporary register is needed (the CPU uses internal temporary storage).

Rules / Notes

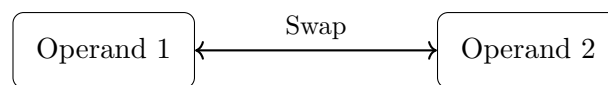
Syntax

```
XCHG operand1, operand2
```

Rules

- Operands: register or memory
- **Only one operand** may be memory (no memory-to-memory)
- **FLAGS are not modified**

Swap Diagram



Example

```
XCHG BX, CX
```

If $BX = 1234H$ and $CX = 5678H$, after swap: $BX = 5678H$, $CX = 1234H$.

Example

```
XCHG AX, [SI]    ; AX <-> word at DS:SI
XCHG AL, BL      ; byte exchange
XCHG BX, [DI]    ; register <-> memory
```

Explanation: Used for fast swapping without extra storage. Also a common exam point: **no FLAGS change**.

4. XLAT (Translate Byte / Table Lookup)

Concept

XLAT performs table lookup. It uses **AL** as an index and **BX** as base address of a table. Output comes back in **AL**.

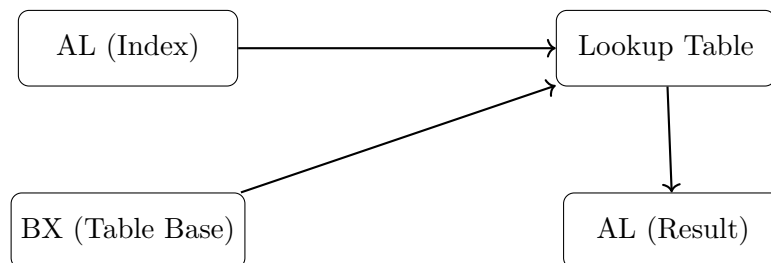
Rules / Notes

No operands. Implicit registers:

- BX = table base (offset)
- AL = index into table
- Result returned in AL
- FLAGS are not affected

$$AL \leftarrow [BX + AL]$$

Concept Diagram



Example

```
DATA SEGMENT
HA_TABLE DB '0123456789ABCDEF'
H_DIGIT DB 07H
ASC_DIGIT DB ?
DATA ENDS

MOV BX, OFFSET HA_TABLE
MOV AL, H_DIGIT
XLAT
MOV ASC_DIGIT, AL
```

Step idea: BX points to table start, AL=07H selects the 8th entry, XLAT loads that byte into AL. Result: *ASC_DIGIT* = '7'.

Exam Tip

Always mention in exams: **XLAT** uses **BX+AL** implicitly and **takes no operands**.

5. LEA, LDS, LES (Pointer Handling)

Concept

These instructions are mainly for **addresses and far pointers**. **LEA** loads only an **offset**, while **LDS/LES** load a **segment:offset**.

5.1 LEA (Load Effective Address)

Rules / Notes

Syntax

```
LEA reg, mem
```

Meaning: loads the **effective address (EA)** of a memory expression into a register.

Important: LEA does **not** read memory data.

Example

```
LEA BX, [SI + 10H]
```

EA is computed: $EA = SI + 10H$ and stored in BX. No memory is accessed.

5.2 LDS (Load Data Segment + Offset)

Rules / Notes

Syntax

```
LDS reg, mem
```

Loads a **far pointer** from memory:

$$reg \leftarrow \text{Offset (word)}, \quad DS \leftarrow \text{Segment (word)}$$

Offset (low word)

Segment (next word)

Example

```
LDS BX, [4326H]
```

Lecture interpretation:

- $BL \leftarrow [DS \times 10H + 4326H]$, $BH \leftarrow [DS \times 10H + 4327H]$
- $DS \leftarrow \text{word at } [DS \times 10H + 4328H : 4329H]$

5.3 LES (Load Extra Segment + Offset)

Rules / Notes

Syntax

```
LES reg, mem
```

Same as LDS but loads into **ES**:

$$reg \leftarrow \text{Offset}, \quad ES \leftarrow \text{Segment}$$

Example

```
LES DI, [5000H]
```

DI receives offset, ES receives segment.

Exam Tip

Fast summary: **LEA** = offset only, **LDS** = offset + DS, **LES** = offset + ES.

Quick Revision

- **MOV**: copies data (no memory-to-memory).
- **XCHG**: swaps values (only one memory operand; FLAGS unchanged).
- **XLAT**: lookup with **BX** base + **AL** index.
- **LEA**: loads effective address (EA), not memory data.
- **LDS/LES**: loads far pointer (offset + segment) into reg and DS/ES.